Table of contents

Introduction to Bash **3**

The terminal, the shell and the prompt . . . 3
`pwd`: Find out where we are . . . 4
`ls`: List the contents of a directory . . . 4
The structure of a command . . . 5
Getting help . . . 6
`mkdir`: Make new folders . . . 7
`touch`: Starting your notebook . . . 9
Writing the notebook in Markdown . . . 9
`cd`: Move around folders . . . 11
Error messages and `Ctrl+C`: Getting unstuck . . . 14
Reading an error message . . . 14
`Ctrl+C`: Stopping a command . . . 15
`wget`: Download data . . . 16
`cp`: Copying data . . . 18
`mv`: Moving (and renaming) data . . . 18
`tar`: Work with tar files . . . 19
`rm`: Remove files and folders . . . 20
Wildcards . . . 21
I/O redirection to new files . . . 26
Exploring the content of text files . . . 27
`head`: View the first lines of a file . . . 27
`tail`: View the last lines of a file . . . 27
`less`: View the full file . . . 28
`wc`: Count things . . . 29
Pipes . . . 29
`cut`: Extract parts of each line . . . 30
`sort` and `uniq`: Create unique lists . . . 32
`cat`: Read and combine files . . . 33
Exploring the content of compressed fastq files . . . 35
`gzip`: (De)compress files . . . 35
`zcat`: Decompress and print to screen . . . 37
`grep`: Find patterns in data . . . 38

`wc`: Count things II 39
`echo` and variables: Storing information 43
For-loops 46
Step 1: Check what the loop works on 46
Step 2: Add the real command 47
`>>`: Collecting the loop output in one file 48
`bash`: Running commands from a script 52
Wrapping up day 1 55

Introduction to Bash

In this first part, we learn to work on your own computer from the command line. On this page, you will:

- Learn what the terminal, the shell and the prompt are
- Move around your computer and create, copy, move and remove files and folders
- Download the sequencing data and look inside it, for example by counting files and lines
- Combine commands with pipes, and repeat a command for many files with a for-loop
- Save your commands in a script, so that you can run them again later
- Start a notebook (`notes.md`) where you write down what you do and why

The terminal, the shell and the prompt

In this tutorial we use the **command-line interface (CLI)** to interact with our computer. It is an alternative to the **graphical user interface (GUI)** that you probably know better. Both let you work with your computer, but in different ways:

- In a GUI, you click on buttons, open folders and use menus.
- In the CLI, you type text to give **commands** (instructions to the computer), and you see the result as text.

When people talk about the CLI, you will hear the words terminal, shell and prompt. They are related, but they are not the same:

- The **terminal** is the window in which you type commands. It is sometimes also called a **console**.
- The **shell** is the program that reads the commands you type in the terminal and tells the operating system what to do. There are several shells, for example **bash** and **zsh**. They use slightly different rules, but the basic commands are the same. This tutorial uses bash.
- The **prompt** is the text that the shell shows when it is ready for your next command. It often shows your user name, the name of your computer and the folder you are in. It usually ends with a `$` sign. For example:

```
user@laptop:~$
```

After you type a command and press Enter, the shell runs it. When the command is done, a new prompt appears and you can type the next one.

`pwd`: Find out where we are

Let's type and run our first command. Begin by opening your terminal and type `pwd`, and then press enter:

```
pwd
```

The command `pwd` stands for *print working directory*. The **working directory** is the folder your shell is operating on right now, so `pwd` tells you where you currently are in the file system. You should see something like this:

```
/Users/YourUserName
```

When you open a new terminal, you start in what is called your **home directory**. This is your personal folder on the computer.

What the path to your home directory looks like depends on your system:

- **Mac:** `/Users/YourUserName`
- **MobaXterm (Windows):** `/home/mobaxterm`
- **WSL2 (Windows) and Linux:** `/home/YourUserName`

In this tutorial, we work inside the home directory. That way, the commands are the same for everybody.

`ls`: List the contents of a directory

Now that you know where you are, let's find out what files and folders are inside this directory. The command `ls` (short for *list*) shows the files and folders in your current directory. Type the following and press enter:

```
ls
```

In my case this returns something like this (and this will look different for your current directory):

The colors and formatting depend on your terminal settings, but typically:

- Folders (directories) appear in one color (in my example above: green)
- Files appear in another (in my example above: bold white)

If your directory contains many items, the output can quickly become overwhelming. To make sense of it, we can use options and arguments to control how commands behave.

The structure of a command

A command generally has three parts:

- A **command name**: the program you want to run, e.g. `ls`
- An **option** (or **flag**): a way to change how the command behaves, e.g. `-l` shows the list in long format, with more details. Options usually start with a `-`
- An **argument**: the input the command works on, e.g. a file or folder. Some commands work without an argument

Try the following command in your current directory to list (`ls`) the contents of the current folder and show details in long format (`-l`):

```
ls -l
```

After running this, we see our files and folders again but as a more detailed list. In the example below you can now see additional information about who is allowed to read and change the files (the access modes that determine exactly who can do what to a file or folder), who owns them, how large the files are, when they were last modified and of course their name:

Tip: Reading the access modes (optional)

Let's look at one line from the example above:

```
drwxrwxrwx 1 ndombrowski ndombrowski 4096 Dec  1 08:51 docs
```

The first column, `drwxrwxrwx`, shows the access modes. Read it in parts:

- The first letter tells you what kind of item it is: `d` for a directory (folder), `-` for a normal file.

- The next nine letters are three groups of three. The first group is for the **owner** of the file, the second for the **group** (a set of users who share access), and the third for **everyone else**.
- In each group, `r` means the user can **read** the file, `w` means they can **write** (change or delete) it, and `x` means they can **execute** it (run it as a program). For a folder, `x` means they can open the folder. A `-` in a position means that this permission is missing.

So `drwxrwxrwx` is a folder that everyone can read, change and open. You will often also see `-rw-r--r--`: a file that the owner can read and change, and that everyone else can only read.
The third and fourth columns show the owner (here `ndombrowski`) and the group.
You don't need to change access modes in this tutorial. They become important when you share data with others, for example on an HPC. If you try to open or change a file that you don't have permission for, you get the error `Permission denied`. If the file or folder belongs to a colleague (check the owner column), you can't fix this yourself. Ask the owner to give you access, or to make a copy for you.

Tip: Using ls in practice

Throughout this tutorial, you'll notice that we use `ls` a lot. There is a reason for that. When working with data, sanity checks are essential, because it is easy to make mistakes, overwrite files, or lose track of where things are. Simple commands like `ls`, `pwd`, `wc` or `grep` help you check what is happening to your files at every step of an analysis.
I still use these checks all the time in my own work.

Getting help

At some point, you'll want to know what options a command has or how it works. In this case, first check if a **manual page** (or *man page*) is available for the command by typing `man` followed by the command name:

```
man pwd
```

This opens the manual entry for the command `pwd`. You can move through it using:

- the ↑ / ↓ arrow keys, or the space bar to move down one page

- `b` to move back up one page
- `q` to quit the manual

MobaXterm users: MobaXterm does not come with manual pages, so `man pwd` does not work. Use `pwd -h` instead (see below), or read the manual page online, for example at man7.org.

Not all programs come with such a manual. Depending on the program, there are a few common ways to get help that you can try:

- `man pwd`
- `pwd --help`
- `pwd -h`

`mkdir`: Make new folders

Before we start moving around, let's first learn how to create new folders (also called directories). This is something we will do often, for example to keep raw data, results and scripts organized in separate places. The command we use for that is `mkdir`, which stands for *make directory*.

We use `mkdir` to create a project folder called `data_analysis` for this tutorial. Don't worry about the `cd ~` in the first line yet, we cover moving around in the next section. For now, it is enough to know that it takes you to your home directory.

```
# Move into the home directory
cd ~

# Create a new folder called data_analysis
mkdir data_analysis

# Check that the folder was created
ls
```

You should see a new folder called `data_analysis` in the list. We will use this folder as our project folder for all exercises in this tutorial.

You can also look at the same folder in the file browser you know from Windows or Mac. The command below opens the folder you are in right now. The dot (`.`) is short for "the current folder":

```
# Windows (MobaXterm or WSL2): open the current folder in File
↪ Explorer
explorer.exe .

# Mac: open the current folder in Finder
open .
```

Use the command for your own system. You should see a window with your home directory and the new `data_analysis` folder in it. This is useful later, for example to open a report that you made with the command line.

> **!** MobaXterm users: did you choose a permanent folder?
>
> If `explorer.exe .` opens a folder in a temporary location, MobaXterm deletes your work when you close it. You can see this in the address bar of File Explorer: it shows `Temp`, for example `Local > Temp > Mxt265 > mx86_64b > tmp > home_YourUserName`. Before you continue, follow the steps in Windows + MobaXterm: choose a permanent folder on the "Before you start" page. Then make the `data_analysis` folder again.

Next, let's make a `data` folder inside the new `data_analysis` folder:

```
# Make a data folder inside the data_analysis folder
# The -p option also makes the parent folder (here: data_analysis) if
↪ it does not exist yet
mkdir -p data_analysis/data

# Check that the folder was created
# Notice how we use ls with an option (-l) and an argument
↪ (data_analysis)?
ls -l data_analysis
```

> Tip: Commenting your code
>
> Notice how I added a `#` and some notes above each command?
> Anything written after a `#` in bash is a **comment**. A comment is not executed, but it helps you (and others) understand what the code does.
> In your own work, add short, meaningful comments above key steps. Avoid restating the obvious. Instead, explain *why* you do something or what it achieves. I add a

comment wherever I think that future me might get confused a month later.

`touch`: Starting your notebook

Now that we have a project folder, let us generate a **notebook**: a file in which you write down the commands you ran, a comment on what each one does, and a few words about what you saw. It works like a lab book for your analysis. Writing it takes a bit of time, but it makes you think about what you just did. It also helps a lot when you come back to the analysis after a few weeks.

We create an empty file for the notebook with `touch`:

```
# Create an empty file called notes.md inside the data_analysis
↪  folder
touch data_analysis/notes.md

# Check that the file was created: it has a size of 0, because it is
↪  still empty
ls -l data_analysis
```

`touch` creates an empty file if it does not exist yet. If the file already exists, `touch` leaves its content alone, so you can not lose your notes by running it twice.

To open the notebook, go to the File Explorer (Windows) or Finder (Mac) window that you opened before, double-click the `data_analysis` folder and then double-click `notes.md`. If it opens in a program other than your text editor, right-click the file, choose **Open with** and pick your editor (for example RStudio).

Tip: RStudio users, stay in Source mode

RStudio shows two buttons at the top left of a Markdown file: **Source** and **Visual**. Stay in **Source**. Then you see exactly what you type, including the `#` and the backticks.

Writing the notebook in Markdown

The file ends in `.md` because we write it in **Markdown**: plain text with a few symbols that give it structure. For your notebook, you only need to know about a few things:

- A line that starts with `#` and a space is a **heading**. More `#` symbols make a smaller heading (`##`, `###`).
- Code goes into a **code block**: a line with three backticks and the word `bash` above the code, and a line with three backticks below it. You can replace `bash` with other language names, like `python` or `r`, based on whatever coding language you use for a task.
- Everything else is normal text, for example what you saw or why you did something.

Notice that a `#` means something different in the two places: in normal Markdown text, it starts a heading. Inside a code block, it is a bash comment.

A first entry in your notebook could look like this:

````
# Notes: Introduction to the command line

## Setting up the project folder

I made a project folder for this workshop in my home directory.

```bash
# Go to the home directory and make the project folder with a data
↪  folder inside
cd ~
mkdir data_analysis
mkdir -p data_analysis/data

# Make the notebook
touch data_analysis/notes.md
```
````

Type this into `notes.md` and save it. From now on, we add to the notebook while we learn new commands.

Markdown can do more, for example lists, links and tables. You don't need these for the workshop, but if you are curious, Markdown Basics on the Quarto website gives an overview.

From now on, you find a blue notebook box at the end of each part of the tutorial. It tells you what to add to your notebook. For the first few, you can compare your notes with mine. Your notes do not have to look the same as mine, as long as future you understands them.

`cd`: Move around folders

Now that we have our own project folder and know how to document what we are doing, let's learn how to move around the file system with `cd` (short for *change directory*).

The file system is structured like a tree that starts from a single **root** directory, written as a forward slash (`/`). All other folders branch out from this root directory. For example, we can go from the root directory to the home folder, and from there into the folder with your user name.

The location of a file or folder is written as a **path**: the folder names along the way, separated by `/`, e.g. `/home/username`. There are two ways to write a path, for example to the data folder:

- An **absolute path** starts from the root, e.g. `cd /home/username/data_analysis/data`
- A **relative path** starts from your current location, e.g. `cd data` if you are already in `/home/username/data_analysis`

It is generally recommended to use relative paths from inside your project folder. That makes your code more portable: it still works if you move the project folder or run it on another computer.

Let's practice moving between folders. At each step, use `pwd` if you feel that you get lost:

```
# Move into the data_analysis folder
cd data_analysis

# Check where you are
pwd
```

You should now see something like `/Users/YourUserName/data_analysis`. You can use `cd` in multiple ways to move around:

```
# Move into the data folder
cd data

# Move one level up in the tree (..), back to the data_analysis
↪  folder
cd ..

# Quickly go back home (you should now be in the home directory)
```

```
cd ~

# Move multiple levels down at once
cd data_analysis/data

# Move two levels up, back into the home directory
cd ../..

# And go back to the data_analysis folder
cd data_analysis
```

In the code above, the tilde symbol (`~`) is a shortcut for your home directory. It is the same as typing the full absolute path to your home (e.g. `cd /Users/YourUserName`), but much faster to type. Two dots (`..`) always mean "the folder one level up".

Tip: Command-line completion

Here are some tips for faster navigation (and fewer typos):

- Use the Tab key for autocompletion: type the first few letters of a folder or file name and press Tab. For example, go to your home directory, type `cd data_` and press Tab. The shell fills in `data_analysis` for you.
- If there is more than one match, the shell can not decide which one you mean. Press Tab twice to see all options.
- Use the ↑ / ↓ arrow keys to scroll through commands you ran before. This is useful to rerun a command or to change a small part of it.

From now on, try to use the Tab key often, so that you do not have to type everything yourself.

Exercise

Get familiar with these first commands:

1. Create a new folder called `results` inside your `data_analysis` folder. We will use it later to store the output of our analyses
2. Move into the `results` folder and confirm your location with `pwd`
3. Move back into the `data_analysis` folder
4. Use `ls` to confirm that both `results` and `data` are there

   Click me to see an answer

```
# Starting from inside data_analysis
mkdir results
cd results
pwd
cd ..
ls
```

Tip: Finding your Desktop and OneDrive folders (optional)

In this tutorial we stay inside the home directory. If you want to work with files that are somewhere else on your computer, for example on your Desktop, these paths help you find them:

Mac

- Your Desktop is at `/Users/YourUserName/Desktop`, or with the `~` shortcut: `~/Desktop`

MobaXterm and WSL2 (Windows)

The terminal reaches your Windows files via the C drive, which is found at `/mnt/c`:

- Your Desktop is usually at `/mnt/c/Users/YourUserName/Desktop`
- Or at `/mnt/c/Users/YourUserName/OneDrive/Desktop` (when using OneDrive)

Accessing the UvA OneDrive folder

If your OneDrive folder name includes spaces (like `OneDrive - UvA`), put quotes around the path. Otherwise bash does not know what to do with the spaces:

```
cd "/mnt/c/Users/YourUserName/OneDrive - UvA"
```

This is also a good example of why you normally do NOT want spaces in your file and folder names.

Be careful with large sequence files in OneDrive folders: OneDrive tries to upload every file you make there.

Error messages and `Ctrl+C`: Getting unstuck

Sooner or later, a command will not do what you expect. That is normal, also for people who use the command line every day. In this section, we look at how to read an error message and how to stop a command that does not finish.

Reading an error message

Let's make a mistake on purpose. From inside your `data_analysis` folder, list a folder with a typo in its name:

```
# A typo on purpose: the folder is called data, not dta
ls -l dta
```

This prints something like this:

```
ls: cannot access 'dta': No such file or directory
```

An **error message** is how the shell tells you what went wrong. Read it from left to right:

- `ls`: the command that ran into a problem
- `cannot access 'dta'`: what it tried to do, and with which file or folder
- `No such file or directory`: why it did not work

The exact wording can be a bit different on your system, but you find the same three parts.

`No such file or directory` is the error you will see most often. It usually has one of three causes:

- There is a typo in the name. Use the Tab key, so that the shell fills in names for you.
- You are in a different folder than you think. Check with `pwd` and `ls`.
- The file does not exist yet, for example because an earlier step did not work.

If you make a typo in the name of the command itself, for example `lss` instead of `ls`, the message contains `command not found`. The shell does not know a command with that name.

When you ask someone for help, always copy the complete error message. With only "it does not work", it is very hard for others to help you.

`Ctrl+C`: Stopping a command

Sometimes a command does not finish, or you start something by mistake. The prompt does not come back, and you can't type a new command. Press `Ctrl+C` (also on a Mac) to stop the command and get your prompt back.

Let's try it. If you give `cat` no file name, it waits for you to type text (we use `cat` for other things later):

```
# cat without a file name waits for input, so the prompt does not
↪  come back
cat
```

Nothing happens and there is no new prompt. Now press `Ctrl+C`. The prompt comes back.

Keep in mind that `Ctrl+C` stops a command, but it does not undo what the command already did.

> Notebook checkpoint: Finding your way
>
> Open `notes.md` and add:
>
> 1. The `mkdir` command from the exercise above, which made the `results` folder, with a comment
> 2. One sentence on the difference between an absolute and a relative path
>
> Save the file.
>
> **Compare with my notes**

````
## Setting up the project folder

I made a project folder for this workshop in my home
↪  directory.

```bash
# Go to the home directory and make the project folder
↪  with a data folder inside
cd ~
mkdir data_analysis
mkdir -p data_analysis/data

# Make the notebook
touch data_analysis/notes.md

# Make a results folder for the output of the analyses
cd data_analysis
mkdir results
```

An absolute path starts at the root `/` and works from
↪  anywhere.
A relative path starts from the folder I am in, so it
↪  only works from the right place.
````

`wget`: Download data

Next, let's use `wget` to download our sequencing data, a file called `seq_project.tar.gz`. Make sure that you are inside your `data_analysis` folder (check with `pwd`), because `wget` saves the file in the folder you are currently in:

```
# Downloaded from GitHub (link provided by course instructor), on
↪  <today's date>
wget
↪  https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.tar
```

In this example, you can also see how I documented my code: I added where the link came

from and when I got it. This helps future me in case I, for example, need to look for some e-mails about the data later on.

! Wget for Mac users

For Mac users `wget` is not installed by default. If you do not want to install it, you can use the `curl` command instead:

```
# -L: if the server says that the file is somewhere else, follow
↪  it until you reach the actual file
# -O: save the file under its original name
curl -LO
↪  https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.ta
```

After you download any kind of data, it is always a good idea to do a sanity check: is the file there, and how large is it? A file size of 0 means that something went wrong during the download.

i Exercise

1. Check with `ls` if the file was downloaded correctly
2. Check the manual for `ls` if there are extra options that we can use for `ls` to see the file size in an easy-to-read unit, such as K (kilobytes) or M (megabytes) (advanced)

   Click me to see an answer

   ```
   # Question 1:
   ls -l

   # Question 2:
   # -h prints the size in a human-readable format
   # Options can be combined: -l and -h become -lh
   ls -lh
   ```

`cp`: Copying data

`cp` (short for *copy*) duplicates files or folders. When working with many files, a good folder structure is essential to keep track of files (useful folders are, for example, `data`, `scripts` and `results`). To keep our project folder organized, let's copy the `seq_project.tar.gz` file we just downloaded into the `data` folder we made earlier:

```
# Copy the tar.gz file into the data folder
cp seq_project.tar.gz data/

# Check what happened
ls -l
ls -l data
```

When running the two `ls` commands, we see that we now have two copies of `seq_project.tar.gz`: one file is in our working directory and the other one is in our data folder. Copying large amounts of data is not ideal, since it uses unnecessary space. However, we can use another command to move the file into our data folder instead.

`mv`: Moving (and renaming) data

`mv` (short for *move*) moves files or folders to a new location without creating a second copy:

```
# Move the tar.gz file into the data folder
mv seq_project.tar.gz data/

# Check what happened
ls -l
ls -l data
```

Notice that `mv` moved the file and, **without asking and without showing**, overwrote the copy that we had put into the data folder with `cp` (we only would know that if we carefully tracked the modification date of the file). So whenever you use `mv`, make sure that you do not overwrite files by mistake.

`mv` is also the command to rename a file: you "move" it to a new name, e.g. `mv old_name.txt new_name.txt`.

Tip: Downloading directly into a folder (optional)

Instead of downloading a file and moving it afterwards, you can tell `wget` directly where to save the file with the `-P` option:

```
wget -P data
↪  https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.ta
```

MobaXterm users: MobaXterm comes with a simpler version of `wget` that does not know the `-P` option. You see the error `wget: Unknown option P`. In that case, download the file first and then move it with `mv`, as we did above.
Mac users: if you use `curl` instead of `wget`, you can save the file into a specific folder with `--output-dir`:

```
curl --output-dir data -LO
↪  https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.ta
```

`tar`: Work with tar files

The data we downloaded is provided as a tar file. A tar file (also called a tarball) bundles many files and folders into one file. It is often also compressed with gzip, which gives the extension `.tar.gz` or `.tgz`.

We can decompress the content of the tarball into our data folder using the following command:

```
tar -xvf data/seq_project.tar.gz -C data
```

The options we use with the `tar` command are:

- `x` tells tar to e**x**tract files from the archive
- `v` prints the name of each file while it is being extracted, so you can see what happens (also called **v**erbose)
- `f` specifies the **f**ile name
- `C` tells tar to change directory (so the package content will be unpacked there)
- `tar` detects the compression automatically when it extracts and we don't need an extra options to tell it to uncompress the file in our command

Tip: How to generate a tarball (optional)

If you ever want to generate a tarball you can do the following:

```
tar -cvzf my_tarball.tar.gz folder_to_tar
```

The options we use are:

- `c` **c**reate an archive by bundling files and directories together
- `z` use g**z**ip compression when generating the tar file

Exercise

1. After extracting the tarball, use `ls` to explore the content of the folder we just extracted. Make sure that you also explore the content of the folders inside it. Hint: use the Tab key (see the tip on command-line completion above), so that you don't have to type every single word.
2. Find the path for at least one sequence file that ends with `fastq.gz`.

   Click me to see an answer

   ```
   # Question 1:
   ls -l data/seq_project

   # Question 2:
   ls -l
   ↪  data/seq_project/barcode004_User2_ITS_9_L001/Sample-DUMMY4_R1.fastq.gz
   ```

`rm`: Remove files and folders

To remove files and folders, we use the `rm` command (short for *remove*). We want to remove the `seq_project.tar.gz` file, since we do not need it anymore once we have extracted the data (and we can download it again if needed). For your own raw data, this is important: Make backups and only delete data if you are absolutely sure that you can recover it from somewhere.

```
# Remove the tar.gz file from the data folder
rm data/seq_project.tar.gz

# Check if that worked
ls -l data
```

If we want to remove a folder, we need to tell `rm` with an option. For this, we use `-r` (short for *recursive*), which removes a folder together with everything inside it.

> ! Important
>
> **Unix does not have an undelete command.**
> This means that if you delete something with `rm`, it's gone. There is no recycle bin. Therefore, use `rm` with care and check what you wrote twice before pressing enter! Also, NEVER run `rm -r` or `rm -rf` (`-f`: "force: don't ask, don't complain") on the root folder `/` or any other important path, such as your home directory. Always double check the path or file name before you press enter when using `rm`.

Wildcards

After we have explored the folder with our sequencing data with `ls`, we have seen that `data/seq_project` contains four folders: two folders for two different users who each generated two samples.

You also might have noticed that it gets tedious to figure out how many files are in each folder, because we would need to run `ls` on each single folder and view its contents one by one. Typing every file name by hand also gets tedious, and you risk typos.

Luckily, the shell provides special characters to work with groups of files using pattern matching. These characters are called **wildcards**. The most useful wildcard is `*`, which stands for "any characters, any number of times (also zero times)". For example, `*.txt` matches all files that end with `.txt`.

We can use `*` to list all files in the `seq_project` folder as follows:

```
ls data/seq_project/*
```

ls data/seq_project/*

will return:

```
data/seq_project/barcode001_User1_ITS_1_L001:
Sample-DUMMY1_R1.fastq.gz
Sample-DUMMY1_R2.fastq.gz

data/seq_project/barcode002_User1_ITS_9_L001:
Sample-DUMMY2_R1.fastq.gz
Sample-DUMMY2_R2.fastq.gz

data/seq_project/barcode003_User2_ITS_1_L001:
Sample-DUMMY3_R1.fastq.gz
Sample-DUMMY3_R2.fastq.gz

data/seq_project/barcode004_User2_ITS_9_L001:
Sample-DUMMY4_R1.fastq.gz
Sample-DUMMY4_R2.fastq.gz
```

The * matches all characters after seq_project/ and therefore will list all folders and the content of each folder

You should have seen before that the sequencing files all end with `.fastq.gz`. With this, we can make the command a bit more specific and at the same time make the output more readable. `*/*.gz` means "inside any folder, any file that ends with `.gz`":

```
ls data/seq_project/*/*.gz
```

ls data/seq_project/*/*.gz

will return:

```
data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
data/seq_project/barcode002_User1_ITS_9_L001/Sample-DUMMY2_R1.fastq.gz
data/seq_project/barcode002_User1_ITS_9_L001/Sample-DUMMY2_R2.fastq.gz
data/seq_project/barcode003_User2_ITS_1_L001/Sample-DUMMY3_R1.fastq.gz
data/seq_project/barcode003_User2_ITS_1_L001/Sample-DUMMY3_R2.fastq.gz
data/seq_project/barcode004_User2_ITS_9_L001/Sample-DUMMY4_R1.fastq.gz
data/seq_project/barcode004_User2_ITS_9_L001/Sample-DUMMY4_R2.fastq.gz
```

The first * matches all characters between seq_project/ and the next /

The second * matches all characters between the last / and .gz

Now we can easily see for each folder how many files we have.

Exercise

1. Use a wildcard to list files for barcode001 only
2. Use a wildcard to only list R1 (forward read) files
3. Use a wildcard to copy all R1 files into a new folder, call this new folder `R1_files`. Use `ls` to confirm if you copied the right files
4. To avoid duplicates, remove the R1_files folder from question 3 after you have ensured the command you used worked. Use `ls` to confirm that your command worked

Click me to see an answer

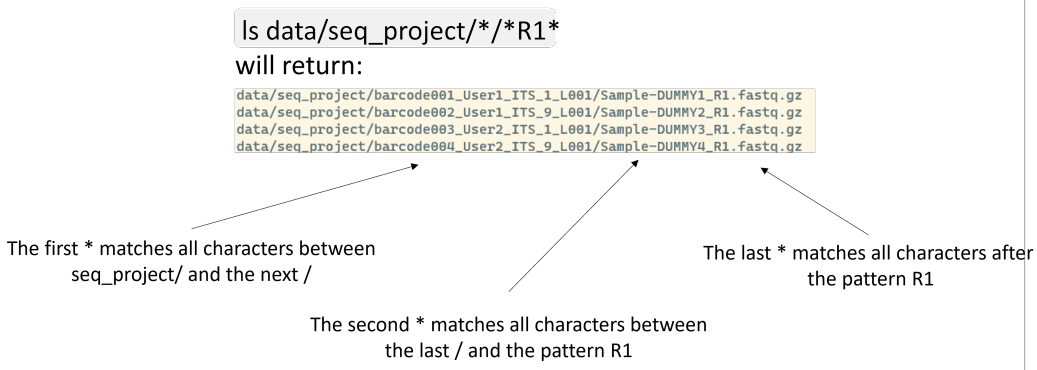
```
# Question 1:
ls data/seq_project/barcode001*/*gz

# Question 2:
ls data/seq_project/*/*R1*

# Question 3:
mkdir R1_files
cp data/seq_project/*/*R1* R1_files
ls data/seq_project/*/*.gz
ls -l R1_files

# Question 4:
rm -r R1_files
```

After solving question 2 you should see:



After solving question 3, we should see that all files still exist in their

original location but are now also inside the `R1_files` folder. Notice how we can use wildcards together with other commands to easily reorganize our data?

Tip: More wildcards (optional)

`*` is not the only wildcard. Another useful wildcard are square brackets, which match exactly **one** character, chosen from a set or a range:

- `[0-9]`: any single digit from 0-9
- `[12]`: a `1` or a `2`
- `[A-Z]`: any single capital letter (`[a-z]` for a small letter)

```
# Folders whose name continues with any digit after barcode00
ls data/seq_project/barcode00[0-9]*/*.gz

# Only the folders of barcode001 and barcode002
ls data/seq_project/barcode00[12]*/*.gz

# Any capital letter in front of 001 at the end of the folder
↪  name
ls data/seq_project/barcode001_User1_ITS_1_[A-Z]001/*.gz
```

You find a full list of wildcards when you open this link.

Tip: Finding files with find (optional)

With `*/*.gz`, `ls` only looks exactly one folder deep inside `seq_project`. If you don't know how deep your files are stored, you can use `find` instead. `find` searches a folder and all the folders inside it, at any depth:

```
# Search data/seq_project and all its subfolders for files ending
↪  in .gz
find data/seq_project -name "*.gz"
```

- `data/seq_project` is the folder where `find` starts searching
- `-name "*.gz"` tells `find` to only print files whose name ends with `.gz`

We put quotes around `"*.gz"` so that the shell passes the `*` to `find` unchanged, and

`find` uses it as a pattern while it searches.
If you want to see the full (absolute) path of each file, start the search from your home directory with `~`:

```
find ~/data_analysis/data/seq_project -name "*.gz"
```

Because the shell replaces `~` with the absolute path to your home directory, `find` now prints absolute paths, and you don't need to type your user name. This works on every system.
Note that `find` does not sort its output alphabetically like `ls` does, so the files may appear in a different order.

ℹ Notebook checkpoint: Getting the data

Add a new heading `## Getting the data` to `notes.md`, and below it:

1. A code block with the commands that got the data into place: download, move, extract, remove the tar file. Leave out the detours, such as the copy we made with `cp` and the `R1_files` folder from the exercise. Someone who runs your code block should end up with the same `data` folder as you
2. In the comment above the download command: where the link came from and on which date you downloaded the data
3. A check that shows what you got, and in normal text below the code block, what it showed: how many folders and how many `.fastq.gz` files

Compare with my notes

````
## Getting the data

```bash
# Download the data
# Downloaded from GitHub (link provided by course
↪  instructor), on 01.11.2026
cd ~/data_analysis
wget
↪  https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_proje

# Move the tar file into the data folder and extract it
↪  there
mv seq_project.tar.gz data/
tar -xvf data/seq_project.tar.gz -C data

# Remove the tar file, we only need the extracted data
rm data/seq_project.tar.gz

# Check: list all sequence files
ls data/seq_project/*/*.gz
```

The data has 4 folders, one per sample (2 users with 2
↪  samples each).
Each folder has 2 files, R1 and R2, so 8 `.fastq.gz`
↪  files in total.
````

I/O redirection to new files

What a command prints is called its **standard output**. We have seen that by default, commands print their standard output directly to the terminal. For example, when we use `ls`, the list of files and folders is printed to your terminal. However, we can also **redirect** the standard output into a file by using the > character.

For example, let's write a list of all fastq files into a file:

```
# Write the list of fastq files into a new file instead of to the
↪  screen
ls data/seq_project/*/* > fastq_paths.txt

# Check that the new file exists and that it is not empty
ls -l fastq_paths.txt
```

In this command:

- `ls data/seq_project/*/*` lists all files in the folders inside `seq_project`
- `>` tells the shell to write this list into a new file called `fastq_paths.txt`, instead of printing it to the screen

> ❗ Important: Be careful with >
>
> The `>` operator overwrites files **without asking**. If a file with the same name already exists, its content is replaced.
> If you want to add (append) to the end of an existing file instead of replacing it, use `>>`.

Exploring the content of text files

Next, let's look inside the list we just made. Looking at the actual content of your files is one of the best ways to catch mistakes early.

`head`: View the first lines of a file

`head` prints the first 10 lines of a file:

```
# Show the first 10 lines
head fastq_paths.txt
```

`tail`: View the last lines of a file

`tail` prints the last 10 lines:

```
# Show the last 10 lines
# Our file has only 8 lines, so both head and tail show everything
tail fastq_paths.txt

# Show the last 3 lines
# The option -n controls how many lines get printed (this also works
↪  for head)
tail -n 3 fastq_paths.txt
```

`less`: View the full file

`less` shows a file one screen at a time. It does not load the whole file at once, so it also opens very large files, such as sequence files, quickly.

```
less -S fastq_paths.txt
```

The option `-S` keeps each long line on one line on the screen, instead of continuing it on the next line. Inside `less`, you can:

- Use the → / ← arrow keys to see the rest of a long line
- Move up and down with the ↑ / ↓ arrow keys, or with Page Up / Page Down
- Press `q` to quit

Tip: Editing text files with `nano` (optional)

You can also change the content of a text file from the command line. The most common tool for this is the text editor `nano`. It comes with WSL2, Mac and most Linux systems:

```
nano fastq_paths.txt
```

Make your changes, and then:

- Press `Ctrl+X` to close the file
- Press `Y` to save the changes, and Enter to keep the file name

MobaXterm users: `nano` is not installed in MobaXterm by default, and you see `nano: command not found`. You can install it with `apt install nano`. Like for `wget` above, it is only kept after a restart if you chose a permanent folder in MobaXterm.

`wc`: Count things

`wc` (short for *word count*) counts the lines, words and characters in a file. With the option `-l`, it counts only the lines. This is very useful for sanity checks. For example, `fastq_paths.txt` has one line per fastq file, so counting its lines tells us how many files we work with:

```
# Count the lines in the file list: one line per fastq file
wc -l fastq_paths.txt
```

We see that we work with 8 files. Here we could easily count that ourselves, but with hundreds of files, `wc -l` is much faster and more reliable.

However, to count the files we first had to make a file with the list. If we do this for every question, we end up with many files that we don't need. In the next section, we see how pipes avoid this.

Pipes

So far, we've run one command at a time. But often, you want to combine commands so that the output of one command becomes the input of the next one. That's what the **pipe** (`|`) does. It allows us to chain simple commands together to do more complex things.

Let's use `ls` to list all fastq files and then pipe the output from `ls` into the `wc` command, to count how many files we work with:

```
ls data/seq_project/*/*.gz | wc -l
```

Here:

- `ls data/seq_project/*/*.gz` lists all .gz files in the folders inside `seq_project`
- The pipe (`|`) sends this list directly to the next command, instead of printing it to the screen
- `wc -l` counts the lines in this list, which is the number of files

Again we see 8 files, but this time without making an extra file first.

Exercise

1. Use `ls` and `wc` to count how many R1 files we have
2. Count how many R2 files we have
3. Count how many files were generated for User1

Click me to see an answer

```
# Question 1: We see 4 files
ls data/seq_project/*/*R1*gz | wc -l

# Question 2: We see 4 files
ls data/seq_project/*/*R2*gz | wc -l

# Question 3: We see 4 files
ls data/seq_project/*_User1_*/*gz | wc -l
```

This exercise also tells us something about our data since we see that for each sample we have a R1 and R2 file. This setup is also called **paired-end** sequencing data: each DNA fragment was read from both ends, which gives one R1 (forward end) file and one R2 (reverse end) file per sample. Having as many R1 as R2 files is a good sanity check: it tells us that we received both files for every sample.

`cut`: Extract parts of each line

Each line in `fastq_paths.txt` is the path to one fastq file, starting from the `data_analysis` folder. For example you might see this line: `data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMM`
What if we only want the file names, without the folders in front of them?

For this, we can use `cut`, which splits each line into parts and prints only the parts we ask for. We tell `cut` where to split with a **delimiter**: a character that separates the parts. In a path, the `/` is a natural delimiter. Each part between two delimiters is called a **field**. If we split our example path at each `/`, we get four fields:

1. `data`
2. `seq_project`
3. `barcode001_User1_ITS_1_L001`
4. `Sample-DUMMY1_R1.fastq.gz`

The file name is the 4th field, so we ask cut for that:

```
# Look at the file list first
head fastq_paths.txt

# Print only the file name: the 4th field when we split at each /
cut -f4 -d "/" fastq_paths.txt

# Do the same, but save the output in a new file
cut -f4 -d "/" fastq_paths.txt > fastq_files.txt

# Check the new file: we expect 8 file names
head fastq_files.txt
wc -l fastq_files.txt
```

Here:

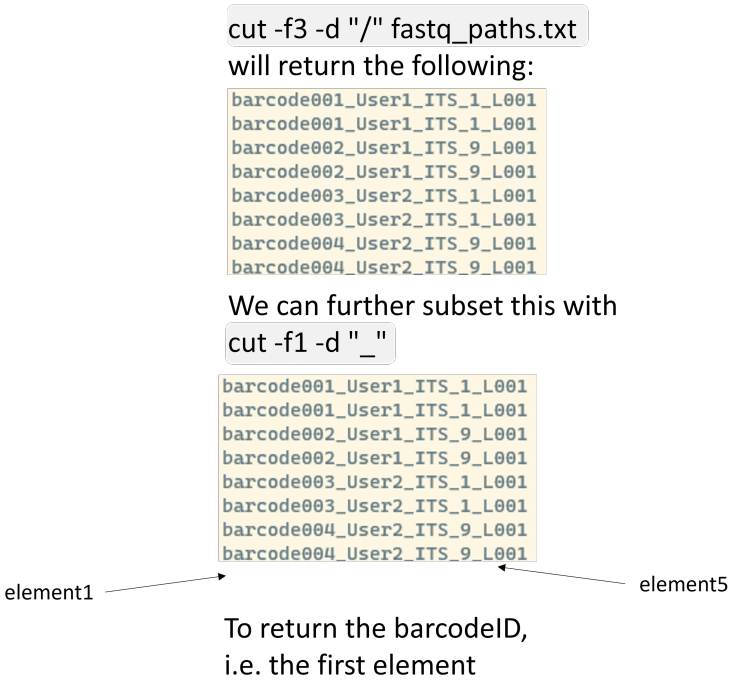
- `-d "/"`: use `/` as the delimiter
- `-f4`: print the 4th field



To extract the file name, we can print only the 4th element with:
`cut -f4 -d "/"`

We can also chain several `cut` commands with a pipe. For example, each folder name, such as `barcode002_User1_ITS_9_L001`, starts with a barcode. To get the barcodes, we first extract the folder name (the 3rd field when we split at `/`), and then get the first part of the folder name (the 1st field when we split at `_`):

```
# Take the folder name (3rd field at /), then the barcode (1st field
↪  at _)
cut -f3 -d "/" fastq_paths.txt | cut -f1 -d "_"
```



This gives us a list of barcodes. But note how each barcode appears twice: once for the R1 file and once for the R2 file. If we want to make a table with information about each sample, we need each barcode only once. For that, we need to learn about two more commands.

`sort` and `uniq`: Create unique lists

- `sort` sorts lines from A to Z
- `uniq` removes duplicate lines, but only when they are sorted directly below each other. That is why we almost always use `sort` before `uniq` to put identical lines next to each other

```
# Extract the barcodes, sorted, with each barcode appearing only once
cut -f3 -d "/" fastq_paths.txt | cut -f1 -d "_" | sort | uniq
```

Exercise

1. Use `cut` on `fastq_paths.txt` to print the names of the folders that contain the fastq.gz files (for example `barcode001_User1_ITS_1_L001`)
2. Print the folder names as in question 1, and then extract the user names from them
3. Change the command from question 2, so that each user name is printed only once

Click me to see an answer

```
# Question 1:
cut -f3 -d "/" fastq_paths.txt

# Question 2:
cut -f3 -d "/" fastq_paths.txt | cut -f2 -d "_"

# Question 3:
cut -f3 -d "/" fastq_paths.txt | cut -f2 -d "_" | sort
↪  | uniq
```

`cat`: Read and combine files

`cat` is short for *concatenate*, which means "link together". We use it for two things:

- Print the content of a file to the screen
- Combine several files into one. You give `cat` the files one after the other, or use a wildcard

To print a file:

```
# Print the whole file to the screen
cat fastq_paths.txt
```

Unlike `less`, `cat` prints the whole file at once. So only use it for short files.

To combine files:

```
# Make a folder for the combined files inside results
mkdir -p results/combined

# Combine two files into one
cat fastq_paths.txt fastq_files.txt >
↪  results/combined/combined_files.txt

# View the new file
cat results/combined/combined_files.txt
```

Notice how we keep the output in its own subfolder of `results`? Giving each analysis its own folder inside `results` keeps the project folder clean, and later you can see right away which files belong together.

Exercise

You can also combine fastq.gz files with `cat`. We don't need this for our data, but you might need it for your own. For example, a sequencing center sometimes sends several files for one sample, because the data is too large for a single file.

1. Use a wildcard and `cat` to combine all R1 fastq.gz files into one file called `combined_R1.fastq.gz`, in the `results/combined` folder. If you are unsure whether your wildcards capture the right files, it is a good habit to first check the behavior with `ls` before concatenating files with `cat`
2. Use `ls` to compare the size of the individual R1 files with the size of the combined file. Did it work?

   Click me to see an answer

   ```
   # Question 1:
   ls data/seq_project/*/*R1.fastq.gz
   cat data/seq_project/*/*R1.fastq.gz >
   ↪  results/combined/combined_R1.fastq.gz

   # Question 2:
   ls -lh data/seq_project/*/*R1.fastq.gz
   ls -lh results/combined/combined_R1.fastq.gz
   ```

   The combined file should be about as large as all the individual R1 files together.

Exploring the content of compressed fastq files

Exercise

1. Use `head` to view the first lines of one of the fastq.gz files
2. Does this look like sequence data to you?

   Click me to see an answer

   ```
   head
   ↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
   ```

   You see a lot of strange characters, but nothing that looks like a DNA sequence. What is going on?

`gzip`: (De)compress files

You might have noticed that our sequence files end with `.gz`. This extension is used for files that are compressed with a tool called `gzip`. Compressing makes a file smaller, which saves space, but it also makes the file unreadable for a human. That is why `head` showed us random characters.

To view the content of a compressed file, we can decompress it first. We do this with the `gzip` command together with the `-d` (decompress) option. Without `-d`, `gzip` compresses a file.

`gzip -d` replaces the compressed file with the decompressed one. We never want to change our raw data in `data/`: if something goes wrong, there is no way to get the original back. So we instead make use of a copy:

```
# Make a folder for this test, so that the raw data in data/ stays as
↪  it is
mkdir -p results/gzip_test

# Copy one sequence file into it
cp
↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪  results/gzip_test/
```

```
# Decompress the copy
gzip -d results/gzip_test/Sample-DUMMY1_R1.fastq.gz

# Check that this worked: the file lost its .gz extension
ls -l results/gzip_test

# View the first lines of the decompressed file
head results/gzip_test/Sample-DUMMY1_R1.fastq
```

Now we finally see DNA sequences, plus some other information. Each sequence that the sequencer read from a DNA fragment is called a **read**. In a fastq file, every read always takes 4 lines:

Exercise

1. Use `ls` with an option to view the file size in an easy-to-read unit (remember the `wget` exercise)
2. Compare the size of the decompressed copy in `results/gzip_test` with the compressed original in `data/`

   Click me to see an answer

   ```
   # The compressed original
   ls -lh
   ↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz

   # The decompressed copy
   ls -lh results/gzip_test/Sample-DUMMY1_R1.fastq
   ```

   We see that the decompressed file has a size of about ~25M, while the compressed file is around ~5M.

   Real sequencing data is usually much larger, so it is best to keep the files compressed.

To compress a file again, use `gzip` without `-d`:

```
# Compress the copy again: it gets its .gz extension back
gzip results/gzip_test/Sample-DUMMY1_R1.fastq
```

```
# Check that this worked
ls -l results/gzip_test
```

We don't need the test folder anymore, so we remove it. Check the path twice before you press Enter. `rm -r` does not ask:

```
# Remove the test folder and everything inside it
rm -r results/gzip_test

# Check that it is gone
ls results
```

`zcat`: Decompress and print to screen

Decompressing a file to look at it works, but it takes extra space and changes the file. `zcat` is a better way: it decompresses the file and prints the content to the screen, but leaves the compressed file as it is. We combine it with `head`, because we only want to see the first lines, not millions of reads:

```
# Print the first 10 lines of a compressed file, without changing the
↪ file
zcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪ | head
```

! `zcat` for Mac users

On a Mac, `zcat` might not work as expected. Try `gzcat` instead:

```
gzcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪ | head
```

`grep`: Find patterns in data

`grep` searches a file for lines that contain a word or a **pattern** (a piece of text you search for), and prints these lines. It is simple, but very useful for sanity checks.

The basic form is `grep "pattern" file`. We put quotes around the pattern, so that special characters, such as `>` or spaces, are passed on to `grep` safely.

For example, to print only the lines for the R1 files:

```
# Print all lines in fastq_paths.txt that contain R1
grep "R1" fastq_paths.txt
```

If you only want to know how many lines match, add the option `-c` (*count*):

```
# Count the lines in fastq_paths.txt that contain R1
grep -c "R1" fastq_paths.txt
```

To look only at the samples from User1, we search for that part of the path:

```
# Print all lines in fastq_paths.txt that contain User1
grep "User1" fastq_paths.txt
```

Exercise

1. Use `grep` to print the paths in `fastq_paths.txt` that contain the pattern "ITS_9"
2. Search for the pattern "AAGACG" in one of the fastq.gz files (advanced)
3. Count how many lines in that file contain "AAGACG" (advanced)

   Click me to see an answer

```
# Question 1
grep "ITS_9" fastq_paths.txt

# Question 2
zcat
 ↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
 ↪  | grep "AAGACG"

# Question 2, but only show the first 10 matching lines
zcat
 ↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
 ↪  | grep "AAGACG" | head

# Question 3
zcat
 ↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
 ↪  | grep -c "AAGACG"
```

For questions 2 and 3, `grep` can't read the compressed file directly, so we send the output of `zcat` to it with a pipe.

In question 2, without `| head` the matching lines scroll past faster than you can read them: thousands of reads contain this pattern. When the output is too long for the screen, add another pipe at the end. `| head` shows only the first lines, `| less -S` lets you scroll through all of them, and `grep -c` (question 3) tells you how many there are.

Commands like these are useful to check whether **adapters** or **primers** are still part of your reads. These are short, known DNA sequences that are added to or used on the DNA before sequencing.

`wc`: Count things II

`wc -l` also works on the output of `zcat`. Since every read takes 4 lines, this tells us how many reads a fastq file has:

```
# Count the lines in one compressed fastq file
```

```
zcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
↪ | wc -l
```

For `Sample-DUMMY1_R2.fastq.gz`, this gives 179,384 lines. Every read takes 4 lines, so the file has 179,384 / 4 = 44,846 reads.

Tip: Better counting (optional)

Dividing by 4 with a calculator works, but bash can also do it for us:

```
echo $(( $(zcat
↪ data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
↪ | wc -l) / 4 ))
```

This command has three parts:

- `$( )` is called **command substitution**. Bash runs the command inside the brackets, here `zcat file | wc -l`, and puts its output in its place. `wc -l` counts the lines that `zcat` sends through the pipe
- `$(( ))` is called **arithmetic expansion**. Bash calculates what is inside the double brackets, here the number of lines divided by 4, and puts the result in its place
- `echo` prints the result

Notebook checkpoint: Looking at the data

Add a new heading `## Looking at the data` to `notes.md`, and below it:

1. A code block with the commands that made the two file lists, `fastq_paths.txt` and `fastq_files.txt`
2. In the same code block, the checks that told you something about the data: how many files there are, whether there are as many R1 as R2 files, what the first lines of a fastq file look like, and how many lines one file has
3. In normal text below the code block: what you learned about the data. How many lines does one read take?
4. Leave out the `gzip` test on a copy, because we removed the test folder again. A short note on why we use `zcat` is enough

Then a check that we come back to: run `ls` and `ls results` inside `data_analysis`.

> Can your notebook explain where every file and folder came from? If not, add a line about it, even if it is “practice file, not needed”.
>
> **Compare with my notes**

````
## Looking at the data

```bash
# Make a list of all fastq files with their paths
ls data/seq_project/*/* > fastq_paths.txt

# Make a list of the file names only, without the
↪  folder names
cut -f4 -d "/" fastq_paths.txt > fastq_files.txt

# Check: we expect 8 files
wc -l fastq_paths.txt

# Check: as many R1 as R2 files
ls data/seq_project/*/*R1*gz | wc -l
ls data/seq_project/*/*R2*gz | wc -l

# Look at the first reads of one file, without
↪  decompressing it
zcat
↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
↪  | head

# Count the lines in one file
zcat
↪  data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
↪  | wc -l
```

There are 8 files, 4 R1 and 4 R2, so every sample has
↪  both.
In a fastq file, each read takes 4 lines: identifier,
↪  sequence, separator (`+`) and quality scores.
Sample-DUMMY1_R2 has 179,384 lines, so 179,384 / 4 =
↪  44,846 reads.

`results/combined/` has the practice files from the
↪  `cat` section. Not needed for the analysis.
````

`echo` and variables: Storing information

Next, we learn how we can use the command line to automate repetitive tasks. But first, we need to learn about two small building blocks.

The first one is `echo`. It prints the text you give it to the screen:

```
echo "Hello"
```

The second one is a variable. A **variable** is a name that stores a value, for example a word, a number or a file path. You create a variable by writing its name, an equals sign and the value:

```
# Store the path to one sequencing file in a variable called
↪  fastq_file
fastq_file=data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
```

This prints nothing, however Bash will remember the value. To use the value, put a `$` in front of the variable name. Bash then replaces `$fastq_file` with the path we stored:

```
# Print the value stored in the variable
echo $fastq_file

# Use the variable in a command, exactly like you would use the path
↪  itself
zcat $fastq_file | wc -l
```

The second command gives the same result as typing out the full path. On its own, that does not save us much work. Variables become useful when their value changes. We use this in the next section on for-loops.

A few rules for variables:

- The name can contain letters, numbers and underscores (_), but no spaces. Choose a name that tells you what is stored in it. `fastq_file` is easier to understand a year later than `x`.
- A variable only exists in the terminal window where you made it. If you close the terminal, it is gone.
- Use lowercase names for your own variables. Bash already uses some names in capitals for its own settings, for example `PATH`, `HOME` and `USER`. If you give your variable one of these names, you replace bash's value, and things stop working.

Important: No spaces around =

Write `fastq_file=data/...` without spaces around the `=`. If you add spaces (`fastq_file = data/...`), bash thinks that `fastq_file` is the name of a command, and you get an error that ends with `fastq_file: command not found`.

Tip: Quotes (optional)

In scripts that you find online, or that an LLMs writes for you, variables are often written in double quotes, for example `"$fastq_file"`. In our examples, the quotes make no difference, because our file names contain no spaces. But quotes change how bash reads the value of a variable, and sometimes that changes the result without any error message.
Let's store the first three names from `fastq_files.txt` in a variable. `$( )` runs the command inside the brackets, and its output becomes the value of the variable:

```
# Store the first 3 lines of fastq_files.txt in a variable
first_files=$(head -n 3 fastq_files.txt)

# Print the variable without and with quotes
echo $first_files
echo "$first_files"
```

This prints:

```
Sample-DUMMY1_R1.fastq.gz Sample-DUMMY1_R2.fastq.gz
Sample-DUMMY2_R1.fastq.gz

Sample-DUMMY1_R1.fastq.gz
Sample-DUMMY1_R2.fastq.gz
Sample-DUMMY2_R1.fastq.gz
```

Without quotes, bash splits the value into separate words at every space and line break, and `echo` prints the words on one line. With quotes, bash keeps the value exactly as it is, including the line breaks.
This matters when the next command works line by line, such as `cut`. Let's extract the sample names:

```
# Without quotes
echo $first_files | cut -f1 -d "_"
```

This prints only one sample name, because `cut` only sees one line:

```
Sample-DUMMY1
```

```
# With quotes
echo "$first_files" | cut -f1 -d "_"
```

This prints one sample name for each of the three files:

```
Sample-DUMMY1
Sample-DUMMY1
Sample-DUMMY2
```

Notice that the version without quotes gives no error. We just get 1 line instead of 3. Mistakes like this are easy to miss, so always have a quick look at the output.
Double or single quotes? Inside double quotes, bash still replaces `$fastq_file` with its value. Inside single quotes, bash changes nothing:

```
echo "$fastq_file"
echo '$fastq_file'
```

This prints:

```
data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz

$fastq_file
```

Quotes and wildcards: inside quotes, bash also does not replace a wildcard with the matching file names:

```
echo "data/seq_project/*/*R1.fastq.gz"
```

This prints the pattern itself, not the four R1 files:

```
data/seq_project/*/*R1.fastq.gz
```

As a rule, we put double quotes around variables, but not around a path with a wildcard. We use single quotes when bash should not change anything at all. You will see all three cases again on day 2.

For-loops

Imagine we want to count the lines not only in one, but in all files. Could we do something like the code below?

```
zcat data/seq_project/*/*.fastq.gz | wc -l
```

When running this, we see that the command prints a single number, 869 944, but not the counts for each individual file, so something did not work right.

The problem with this command is that it prints the text from all 8 fastq files and only afterwards counts the lines. We basically combined all files and then counted the lines of all 8 files together. However, what we want is to repeat the steps below, individually for each file:

- Decompress a first file
- Count the lines in the first file
- Decompress a second file
- Count the lines in the second file
- ...

A **for-loop** repeats one or more commands, once for each item in a list. Each round of the loop is called an **iteration**.

Step 1: Check what the loop works on

Let's write a first loop that does not change anything yet. It only prints the names of our sequencing files:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo $fastq_file
done
```

This prints the paths of all 8 fastq files, one per line. Let's go through the parts of the loop:

- `for fastq_file in data/seq_project/*/*.fastq.gz; do`: this line says what to repeat over. The wildcard gives the list, here all 8 fastq files. In each iteration, bash stores the next file from the list in the variable `fastq_file`. You choose the name of this variable yourself.

- `echo $fastq_file`: the command we want to repeat, written between `do` and `done`. Here, we print the value of the variable. We indent this line with 4 spaces. Bash does not need this, but it makes the loop easier to read.
- `done`: marks the end of the loop.

In other words, the loop does the same as typing these commands one after the other:

```
# You don't need to run this, it only shows what the loop does for us
fastq_file=data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R1.fastq.gz
echo $fastq_file
fastq_file=data/seq_project/barcode001_User1_ITS_1_L001/Sample-DUMMY1_R2.fastq.gz
echo $fastq_file
# ... and so on, for the other 6 files
```

Starting a loop with `echo` is a good habit. This is called a **dry run**: the loop shows what it would work on, without changing anything. Before you add the real command, check the list. Do you see all 8 files, and only the files you expect? If yes, then you can continue with the next step and build up the loop for more complex tasks.

> Tip: Typing a loop in the terminal
>
> When you press Enter after writing the first line of the loop, nothing runs yet. Instead, the prompt changes to `>` (this `>` is only the prompt, it doesn't write anything). This means bash waits for the rest of the loop. It only starts once you type `done` and press Enter. (On a Mac with zsh, this prompt may look slightly different, for example `for>`.)
> If you get stuck in this `>` prompt, for example because of a typo, press `Ctrl+C` to cancel and start again.

Step 2: Add the real command

Now that we know the loop works on the right files, we can replace `echo` with the command we actually want to run:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    zcat $fastq_file | wc -l
done
```

Now we get 8 numbers, one per file. `zcat $fastq_file | wc -l` is the same command as in the variables section. The only difference is that bash now runs it 8 times, each time with a different file stored in `fastq_file`.

Tip: Loops written on one line (optional)

You might also see loops written on a single line, with `;` where we start a new line:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do zcat
↪  $fastq_file | wc -l; done
```

This does exactly the same as the loop above. For anything longer than one short command, the version over several lines is easier to read.

Exercise

1. Use a for-loop and count how often grep finds the pattern "TAAGA" in each individual fastq.gz file

   Click me to see an answer

   ```
   for fastq_file in data/seq_project/*/*.fastq.gz; do
       zcat $fastq_file | grep -c "TAAGA"
   done
   ```

>>: Collecting the loop output in one file

So far, the counts only appear on the screen. We cannot see which number belongs to which file, and the numbers are gone once we close the terminal. Let's write the file names and the counts into a file instead.

In the section on I/O redirection, we have seen that `>` replaces the content of a file, and that `>>` adds (**appends**) text to the end of a file. Inside a loop, this difference matters. With `>`, each iteration would replace what the iteration before it wrote, and only the result for the last file would be left. So inside the loop we use `>>`:

```
# Make a folder for the counts inside results
mkdir -p results/counts

# Start the counts file with a first line that describes its content
# Because we use > here, an existing file is emptied first, so we
↪  start fresh each time
echo "Number of lines per fastq file" >
↪  results/counts/line_counts.txt

# For each fastq file, add the file name and its number of lines to
↪  the end of the file
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo $fastq_file >> results/counts/line_counts.txt
    zcat $fastq_file | wc -l >> results/counts/line_counts.txt
done

# Check the result: we expect 1 header line + 2 lines for each of the
↪  8 files = 17 lines
wc -l results/counts/line_counts.txt
cat results/counts/line_counts.txt
```

Here:

- `mkdir -p results/counts`: we keep the output of each analysis in its own folder inside `results`
- `echo "Number of lines per fastq file" > results/counts/line_counts.txt`: before the loop starts, we write a first line that describes what is in the file. Such a line is called a **header**. Because we use `>` here, this also empties the file if it already exists from an earlier run
- Inside the loop, there are now two commands. The first one adds the name of the file, the second one adds its number of lines. Both use `>>`, so each iteration adds two lines to the end of `line_counts.txt`

We will use these numbers again on day 2, to check what a quality filtering tool did to our data.

> **Exercise**
>
> 1. Run only the for-loop from the code above a second time, without the `echo` line before it. How many lines does `line_counts.txt` have now? Why?

2. Now run the complete code again, including the `echo` line before the loop. How many lines does `line_counts.txt` have now?
3. What do you think would happen if we used `>` instead of `>>` inside the loop? Think about it first, then try it. Afterwards, run the code with `>>` once more, so that `line_counts.txt` is correct again.

Click me to see an answer

1. The file now has 33 lines. The loop added its 16 lines a second time to the end of the file, so all counts appear twice. This is something to watch out for whenever you use `>>`.
2. The file has 17 lines again. The `echo` line uses `>`, which empties the file before the loop adds the counts.
3. Each `>` replaces the content of the file. After the loop, the file only contains a single line: the number of lines of the last file. All other results were overwritten.

Tip: Better counting in for loops (optional)

In our counts file, each file name and its count end up on two separate lines. With 100 files, it is much easier to read if the file name and its count are on the same line. We can do this with the command substitution from the tip "Better counting" in the section "`wc`: Count things II":

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo "$fastq_file: $(zcat $fastq_file | wc -l)"
done
```

Here:

- `echo "..."` prints the text between the double quotes. Inside double quotes, bash still replaces `$fastq_file` with the file name, and `$( )` with the output of the command inside it
- So for each file, `echo` prints the file name, a colon, a space and the number of lines

To get the number of reads, we also divide by 4 with the arithmetic expansion `$(( ))`:

```
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo "$fastq_file: $(( $(zcat $fastq_file | wc -l) /4 ))"
done
```

Notice the single and the double brackets: `$( )` runs a command, `$(( ))` calculates.

Tip: Using bash to make sample mapping files (optional)

Many tools for amplicon analysis need a table that lists your files, often with the R1 and the R2 file of each sample in separate columns. This is called a **sample mapping file**. Let's have a look at how we could automatically make one with bash.
The code below uses a few things we have not covered. We won't go into every detail but the goal is to show you what is possible with bash.

```
for file in data/seq_project/*/*.gz; do
    if [[ $file == *_R1.fastq.gz ]]; then
        r1_file=$file
        r2_file=$(echo $file | sed 's/_R1/_R2/')
        echo "$r1_file,$r2_file"
    fi
done
```

Here:

1. `for file in data/seq_project/*/*.gz`: loop over all fastq.gz files
2. `if [[ $file == *_R1.fastq.gz ]]; then ... fi`: run the lines between `then` and `fi` only if the file name ends in `_R1.fastq.gz`. So the loop skips the R2 files
3. `r1_file=$file`: store the path of the R1 file in the variable `r1_file`
4. `r2_file=$(echo $file | sed 's/_R1/_R2/')`: the command `sed` replaces `_R1` with `_R2` in the path. This gives the path of the matching R2 file, which we store in the variable `r2_file`
5. `echo "$r1_file,$r2_file"`: print both paths on one line, separated by a comma

We can add a column with the name of the sample folder, with `cut` from above:

```
for file in data/seq_project/*/*.gz; do
    if [[ $file == *_R1.fastq.gz ]]; then
        sample_name=$(echo $file | cut -f3 -d "/" )
        r1_file=$file
        r2_file=$(echo $file | sed 's/_R1/_R2/')
        echo "$sample_name,$r1_file,$r2_file"
    fi
done
```

Or, with a second `cut`, we could only add the barcode:

```
for file in data/seq_project/*/*.gz; do
    if [[ $file == *_R1.fastq.gz ]]; then
        sample_name=$(echo $file | cut -f3 -d "/"  |cut -f1 -d
        ↪  "_" )
        r1_file=$file
        r2_file=$(echo $file | sed 's/_R1/_R2/')
        echo "$sample_name,$r1_file,$r2_file"
    fi
done
```

`bash`: Running commands from a script

Our counting loop is now a few lines long, and we might want to run it again later, for example when new data arrives. Instead of typing or copying the loop each time, we can save it in a file. A text file that contains commands is called a **script**. Bash scripts usually end with `.sh`.

To keep our project folder tidy, we store scripts in their own folder:

```
# Make a folder for scripts inside data_analysis
mkdir -p scripts

# Check that the folder was created
ls -l
```

Next, we create the script with the text editor `nano` (see the tip Editing text files with `nano` above):

```
nano scripts/count_lines.sh
```

MobaXterm users: if you see `nano: command not found`, install it first with `apt install nano`, as explained in that tip.

Type or copy the following into the editor:

```
#!/bin/bash

# Count the number of lines in each fastq file
# Run this script from inside the data_analysis folder

# Make a folder for the counts and start the counts file with a
↪ header
# We give this output a new name, so you can compare it with the file
↪ from the loop
mkdir -p results/counts
echo "Number of lines per fastq file" >
↪ results/counts/line_counts_bash.txt

# For each fastq file, add the file name and its number of lines to
↪ the end of the file
for fastq_file in data/seq_project/*/*.fastq.gz; do
    echo $fastq_file >> results/counts/line_counts_bash.txt
    zcat $fastq_file | wc -l >> results/counts/line_counts_bash.txt
done
```

Save and exit:

- Press `Ctrl+X` to exit (indicated with `^X` in nano), then `Y` to confirm that you want to save the file, then Enter to keep the file name

The commands are the same as in the `>>` section. Two things are new:

- `#!/bin/bash`: this first line is called the **shebang**. It tells the system that the script should be run with bash. Put it at the very top of every bash script.
- The lines that start with `#` are comments. Bash ignores them, but they tell you (and others) what the script does and how to use it.

Now we can run the whole script with a single command:

```
# Run the script with bash
bash scripts/count_lines.sh

# Check the output: we again expect 17 lines
wc -l results/counts/line_counts_bash.txt
cat results/counts/line_counts_bash.txt
```

The script itself prints nothing to the screen, because all output goes into `line_counts_bash.txt`. That is why we check the file afterwards.

Saving your commands in a script has a few advantages. You can run the same analysis again, with exactly the same commands and without typing mistakes. Together with your notebook (`notes.md`), the script also documents how you made a result. On day 2, we use exactly this kind of script on the HPC. There, we add a few extra lines at the top that tell the HPC how many resources our job needs.

! Important: Write scripts in nano or a code editor, not in Word or Notepad

Windows programs such as Notepad end every line with an extra invisible character. Bash on Linux does not understand this character, and the script fails with errors such as `$'\r': command not found` or `syntax error near unexpected token`. Word processors such as Word also add formatting and can change straight quotes (`"`) into curly quotes.
To avoid this, write your scripts with `nano`. If you prefer a code editor such as RStudio or VS Code, check that it saves the file with `LF` line endings (in VS Code, you see `LF` or `CRLF` in the bar at the bottom right, and can click it to change it). This becomes important on day 2, where the scripts run on a Linux computer.
If you like writing in RStudio, you can write the script there and copy it into `nano`. Or set RStudio to save with `LF` line endings: in **Tools > Global Options > Code > Saving**, set **Line ending conversion** to **Posix (LF)**.

i Notebook checkpoint: Counting lines per file

Add a new heading `## Counting lines per file` to `notes.md`, and below it:

1. The command that ran the script, with a comment that says what the script does and which file it makes. We use `results/counts/line_counts_bash.txt` again on day 2. You don't need the code that made `line_counts.txt`, because it has the same content. One line that says where it came from is enough.

2. The check you ran afterwards, and below the code block, what you expected and what you saw

You don't need to copy the loop itself into your notebook, because it is saved in `scripts/count_lines.sh`. The script is part of your documentation. Your notebook only needs to say which script you ran and what it made.

Wrapping up day 1

At the end of day 1, your project folder contains the data, some results, a script and your notebook. Before you close the terminal, take a few minutes to read through your notebook.

Notebook checkpoint: Tidy your notebook

1. Below the main heading at the top of `notes.md`, add 2 or 3 sentences: what data this is, where it came from, and what you did with it today
2. Read your notebook from the top. Could you redo today's analysis from it alone, without this tutorial? Add what is missing
3. Check once more that your notebook explains every file and folder in your project. Run these inside `data_analysis`: `ls`, `ls results/*` and `ls scripts`
4. Add a heading `## Questions for day 2` and write down anything that is still unclear. This is also handy for a real analysis and lets you keep track of loose ends.

On day 2, you keep writing in the same `notes.md` on your own computer, also for the commands that you run on the HPC.